UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Laboratorio de Ciberseguridad · Semillero BlackByte
OctoMatrix — Detección de ataques OWASP en tráfico HTTP (Informe reproducible)
Docente: Biviana Yulieth Aguirre Arias · Integrante: Santiago Potes Giraldo

Resumen

OctoMatrix es un detector de anomalías OWASP sobre peticiones HTTP. Esta versión corrige el prototipo inicial, que se entrenaba con **72 ejemplos escritos a mano** y reportaba un 83% de precisión sobre un test de 15 muestras. Aquí se entrena con **~75.500 peticiones reales** (CSIC 2010, ECML/PKDD 2007 y los tests de regresión del OWASP CRS) y se evalúa con métricas propias de un WAF. El sistema es un **Mixture-of-Experts por procedencia de datos**: un experto sobre datos reales, uno sobre payloads generados (cobertura OWASP) y uno sobre flujos de red. Este informe prioriza la **reproducibilidad**: la sección 6 replica todo el experimento en una máquina limpia con un solo comando.

1. Objetivo

Clasificar tráfico HTTP como *normal* o *ataque* con un umbral calibrado a una tasa de falsos positivos (FPR) objetivo, sirviendo como capa heurística para un WAF o un monitor de tráfico, y medir honestamente su capacidad de generalización.

2. Materiales

Python 3.13, scikit-learn, pandas, numpy, pyarrow. Sin GPU. Fuentes de datos públicas, descargadas bajo demanda (no se redistribuyen):

Fuente	Rol	Aporte
CSIC 2010	tráfico real de una tienda web	normal + anómalo (sin tipo)
ECML/PKDD 2007	tráfico sintético etiquetado	7 tipos de ataque
OWASP CRS (coreruleset)	tests de regresión de reglas	SQLi, XSS, RCE, LFI, ...
CSE-CIC-IDS2018	flujos de red (CICFlowMeter)	DoS, fuerza bruta (experto de flujos)

Tabla 1. Fuentes de datos.

3. Metodología

Pipeline por etapas: **Generar/Recolectar** → **Limpiar** → **Particionar** → **Entrenar** → **Probar** → **Exportar** → **Post-test**. Un único *sklearn.Pipeline* por experto: TF-IDF de n-gramas de caracteres + características de seguridad (regex + entropía de Shannon) + Random Forest, con probabilidad **calibrada** (CalibratedClassifierCV), un **post-filtro regex** que reduce falsos positivos, y un umbral elegido en validación para un FPR objetivo. El artefacto se serializa con joblib + verificación sha256 (no pickle suelto).

Partición sin fugas: las peticiones casi-duplicadas se agrupan y van completas al mismo lado; además se evalúa **entre fuentes** (entrenar con unas, probar con otra) para medir generalización real, no memorización.

4. Resultados reales

Escenario	Qué mide	ROC-AUC	recall@FPR 1%	recall@FPR 0,1%
in_source	dentro de las fuentes vistas	0,989	89,6%	59,1%
cross-source	generalización a fuente NUEVA	0,53	—	—
adversarial (MoE)	cobertura 19 cat. OWASP	—	~100%	—
flujos (CIC-IDS2018)	DoS / fuerza bruta	1,00	100%	100%

Tabla 2. Métricas honestas por escenario (modelo real, umbral por FPR).

Lectura honesta: dentro de las fuentes conocidas el detector es fuerte (recall 89,6% molestando a 1 de cada 100 usuarios legítimos). Ante un **sitio nuevo**, ROC-AUC 0,53 (azar): aún no generaliza, por falta de diversidad de tráfico normal. El 1,00 de flujos refleja que CIC-IDS2018 es trivialmente separable, no que esté resuelto en producción. El experto generado sube la cobertura OWASP a ~100% **sin** alterar al experto real (aislamiento verificado: FPR extra +0,00%).

5. Latencia

Experto de texto: ~39 ms por petición suelta, ~2 ms/petición en lote. Experto de flujos: ~0,05 ms/flujo en lote. Apto para un consumidor Kafka que procesa por lotes.

6. Replicación desde cero (un comando)

En una máquina limpia (incluida una instancia cloud), todo el experimento se reproduce copiando estas líneas en una terminal:

```
git clone https://github.com/SPotes22/OctoMatrix
cd OctoMatrix
bash reproducir.sh # texto (experto real + generado)
WITH_FLOWS=1 bash reproducir.sh # además, experto de flujos (~0,8 GB)
```

El script *reproducir.sh* crea el entorno, descarga las fuentes, unifica/limpia/particiona, entrena, ensambla el MoE y evalúa. Es idempotente: si un insumo ya existe, no lo rehace. Los datos crudos no se versionan; un MANIFEST.json fija el commit/sha256 exacto de cada fuente.

7. Limitaciones conocidas

• No generaliza todavía a sitios no vistos: necesita tráfico normal variado (logs propios vía --own-logs).

• El set adversarial es sólo evaluación: entrenar con él inflaría el resultado (memorización).

• CSIC marca como anómalas peticiones ilegales pero no maliciosas: hay ruido de etiqueta.

• IDOR y escalada de privilegios son poco detectables sólo por el payload.

• No reemplaza validaciones determinísticas ni otros controles de seguridad; es una capa heurística.

8. Referencias

[1] HTTP DATASET CSIC 2010. Information Security Institute, CSIC.

[2] ECML/PKDD 2007 Discovery Challenge — Analyzing Web Traffic.

[3] OWASP Core Rule Set — regression tests. github.com/coreruleset/coreruleset

[4] CSE-CIC-IDS2018. Canadian Institute for Cybersecurity, UNB.